

26

+

Inside a Small Game Engine

An Architecture Tour in Modern C++

ELIAS FARHAN



20
26



September 12 - 18

Elias Farhan

Head of Code & Tech at Geneva

School of Digital Arts

Head of Games Programming at

SAE Institute Geneva

Co-Founder of indie game studio

Team Kwakwa

Co-Organizer of the game
development meetup in

Lausanne



What this talk is

- Some good patterns to make a game engine in C++
- What to do after making a small C++ game

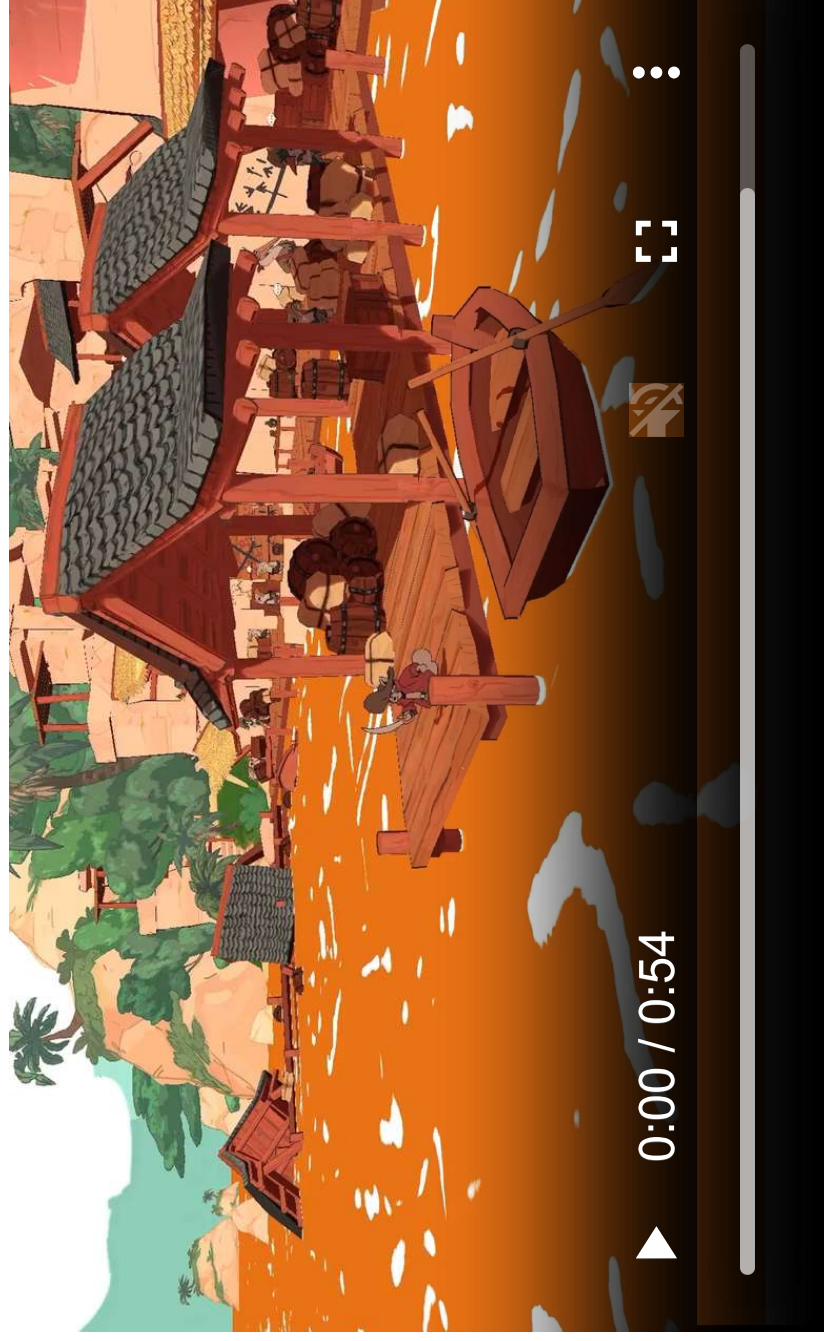
NOT IN SCOPE

- Making a full AAA game engine
- Going too much into implementation details

The tour

1. What's a custom game engine?
2. Containers
3. Index-based Pointers
4. Scene abstraction
5. Rendering
6. Asset loading
7. Serialization

A very emotional talk



My summer goal

Port *Soup Raiders* to a custom C++ engine

Nintendo Switch port at 60 Hz

Sub-3s loading.

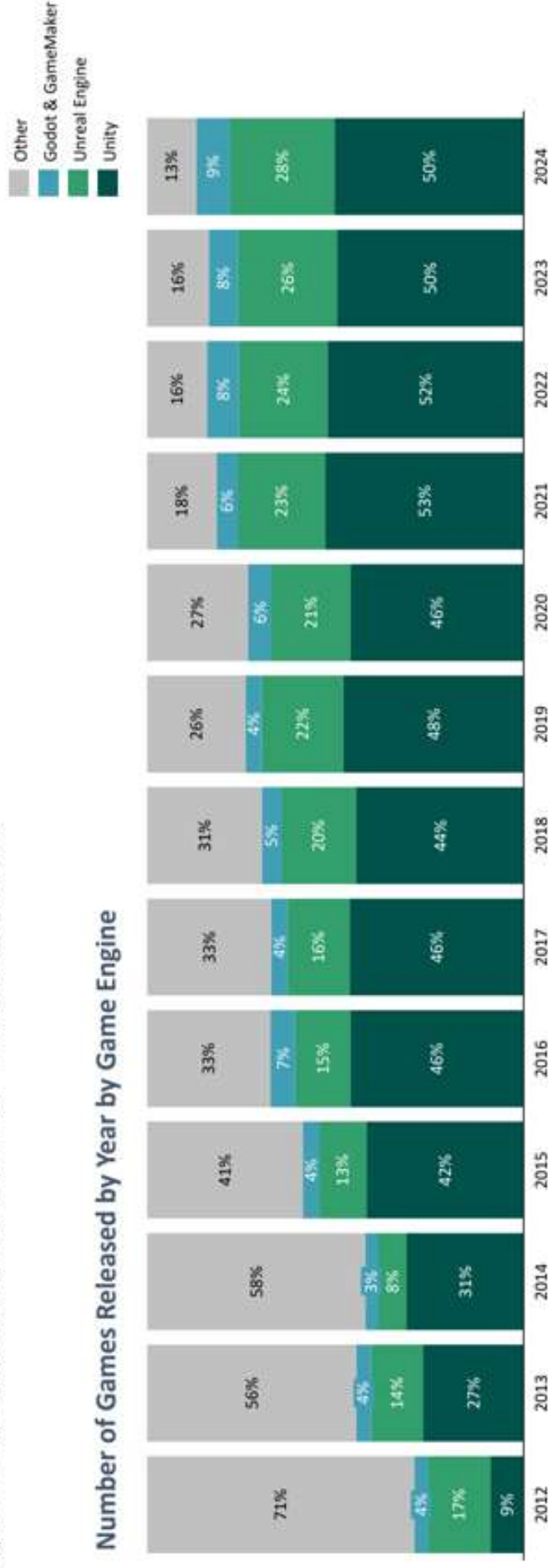


Custom Game Engine Share

13% of games released on Steam in 2024

But 71% in 2012.

Game Engine Market Share on Steam Over Time - # of Games Released

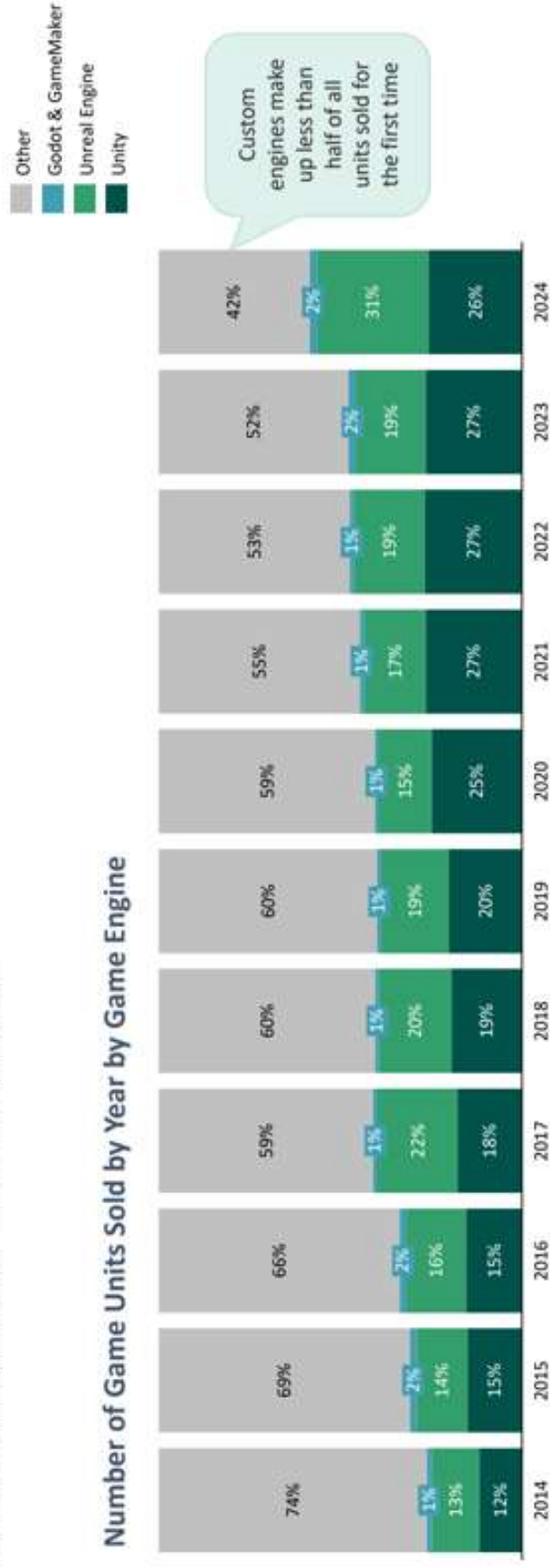


Source: VGInsights

42% of games sold on Steam in 2024

But 74% in 2012.

Game Engine Market Share on Steam Over Time - # of Units Sold



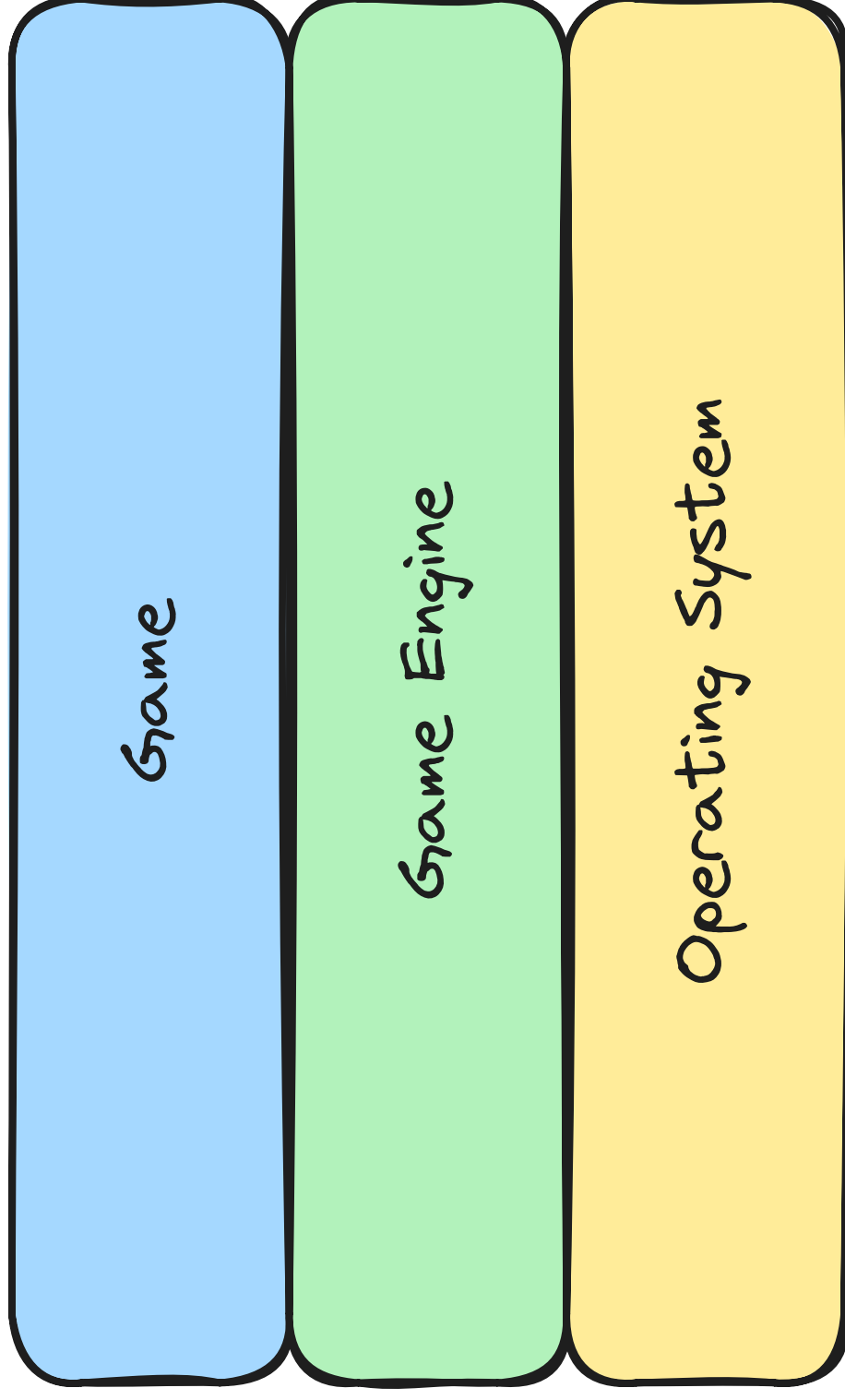
Source: VGInsights

Some lovely indie games have their own custom engines

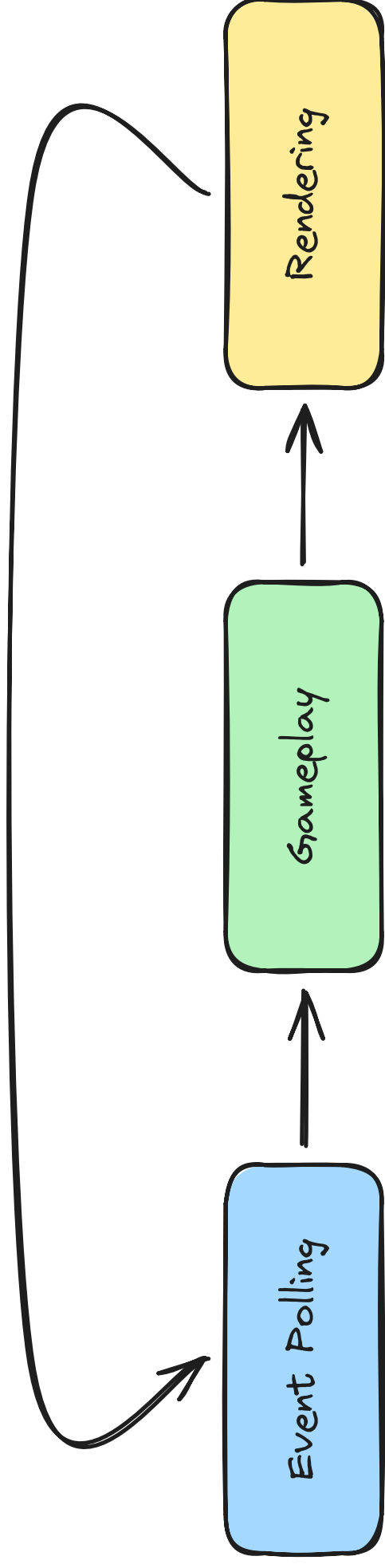
- Factorio, Noita, Stardew Valley
- Darkest Dungeon, Shovel Knight, Don't Starve
- Into the Breach, FTL, Banished
- Prison Architect, Crypt of the NecroDancer
- Bastion, Starbound, Legend of Grimrock

What do we mean by custom game engine?

Game stack



Game loop



And the rest...

- Meshes
- Animations
- Sprites
- Audio
- Fonts
- Scenes
- Serialization

Framework vs game engine

▲ 20 days ago | prev | next [-]

Where do you put modular engine/framework hybrids like Bevy? It owns the game loop so it's technically an engine but you can swap out just about every part of it, even rendering and window management. Tiny Glade only used the core ECS with every other part being custom.

▲ 20 days ago | parent | next [-]

Framework, Bevy is definitely not an engine.

Does it do one thing? It's probably a library.

Is it a collection of libraries that work together focusing on primitives with callbacks? It's probably a framework.

Does it load levels? Have its own schema for an entity? Provide a multiplayer server protocol, and build on primitives? It's probably an engine.

Libraries -> frameworks -> engines -> games.

▲ 20 days ago | root | parent | next [-]

Bevy is very much an engine. It's an just an engine without an editor.

A really, really well designed engine is just going to be a library of highly modular components with a nice editor or two thrown in.

As the old adage goes "Build libraries not frameworks"

▲ 20 days ago | root | parent | prev | next [-]

Bevy is definitely an engine. An extremely modular one. I'm currently building a game with it right now and having a blast.

HN on deciding if Bevy is a Framework or an Engine

Framework vs game engine

DEFINITION

Do your tools allow you to assemble, author, run, and manage your game?

→ Then it's a game engine.

→ If not, it's a framework.

Is it custom?

- Unity + a lot of internal tools →
- SDL3 + Assimp for a 3D game →
- Unreal, but I rewrote the renderer →
- SFML for a 2D networked game →
- Godot extended in C++ →
- DX12 by hand + Win32 window →

Engine core

Some standard containers

- `std::array` usually allocates on the stack and it's full (no `push_back` for example).
- `std::vector` allocates on the heap and empty is a state.

But what if I want to have the same behavior as `std::vector`, but I know the maximum capacity and it's small?

Red Bearon Bombs



The Red Bearon throwing bombs at our crew

std::inplace_vector

```
inline constexpr size_t kMaxBombsCount = 6;  
std::inplace_vector< Bomb, kMaxBombsCount > bombs_;
```

- But what happens when we `push_back` beyond the capacity? →
- I prefer using `try_push_back` that returns →

neko::SmallVector

- Because of current availability only on GCC 16, I have my own

```
neko::SmallVector < class T , size_t N >
```

```
inline constexpr size_t kMaxBombsCount = 6;  
  
neko::SmallVector< Bomb, kMaxBombsCount > bombs_;
```

Dialog System

```
<< declare $talkToVeteran = false as bool >>  
[...]  
<< set $talkToVeteran to true >>
```

This will call

```
DialogRuntime::SetSessionVariable("$talkToVeteran", 1);
```

Dialog system variables

```
"variables": {  
  "$beatDupont": 0,  
  "$beatSeagulls": 0,  
  "$beatBearon": 0,  
  "$beatTutorial": 0,  
  "$failComboPush": 0,  
  "$comboPushPosition": 0,  
  "$passComboPush": 0,  
  "$pushTutorial": 0,  
  "$dupontSeagullsDone": 0,  
  "$dupontLost": 0,  
  "$tutorialPhase": 0,  
  "$tutorialDone": 0,  
  "$tutorialSuccess": 0,  
  "$stalkToVeteran": 0  
},
```


What about using maps?

- `std::map` : red-black tree of key-value pairs
- `std::unordered_map` : hash-table key/value container, unordered; $O(1)$ average lookup/insert
- `std::flat_map` : `std::vector` of keys and `std::vector` of values.

They all allocate on the heap by default. What about a small map?

neko::SmallFlatMap

```
using SmallFlatMap =  
    std::flat_map<  
        Key,  
        Value,  
        std::less< Key >,  
        neko::SmallVector< Key, N >,  
        neko::SmallVector< Value, N >>;  
  
SmallFlatMap< std::string, int, 16>;
```

Is everything on the stack?

Key	Length
\$bearonDorothyIntroPass	23
\$dupontSeagullsDone	19
\$comboPushPosition	18
\$tutorialSuccess	16

Using `neko::FixedString`

A stack-based fixed size string:

```
neko::SmallFlatMap< neko::FixedString < 24 >, int, 16 > sessionVars_;
```

Index-based pointer

```
using Index = size_t;
std::vector<Node> nodes_;

Index CreateNode()
{
    const auto newNodeIndex = nodes_.size();
    nodes_.emplace_back();
    return newNodeIndex;
}
```

Source of bugs

```
using Index = int32_t;

struct SceneNodeInfo
{
    Index parent = -1; // index to other node
    Index first_child = -1; // index to other node
    Index next_sibling = -1; // index to other node
    Index draw_command_index = -1;
    Index subpass_index = -1;
    Index mesh_index = -1;
    ...
};
```

Strongly typed index

```
// instead of using TextureId = int;
enum class TextureId : int
{
};
constexpr TextureId kInvalidTextureId = TextureId{-1};
//Somewhere else
const Texture& TextureManager::GetTexture(core::TextureId textureId) const
{
    if(textureId == kInvalidTextureId)
```

Removing an element?

```
std::vector < Node > nodes_;

void RemoveNode(Index removedIndex)
{
    // This is a bad idea, because the next elements will have wrong indices
    nodes_.erase(nodes_.begin() +
        static_cast< size_t >(removedIndex));
}
```

The answer is to invalidate the Index, not to remove the element from the `std::vector`.

Generational Index

```
template < typename T, typename IndexType=int,  
         typename GenerationIndexType=int >  
class Index  
{  
    IndexType index_ = -1;  
    GenerationIndexType generationIndex_ = 0;  
};
```

Check the validity of the generational index

```
std::vector < Node > nodes_;  
std::vector < int > nodesGenerations_;  
  
std::optional< Node& > GetNode(Index index)  
{  
    if(index.generation() != nodesGenerations_[index])  
    {  
        return std::nullopt;  
    }  
    return nodes_[index.index()];  
}
```

Removing an element

```
std::vector < Node > nodes_;
std::vector < int > nodesGenerations_;

void RemoveNode(Index removedIndex)
{
    if(index.generation() == nodesGenerations_[index])
    {
        //Only a valid index can be removed
        ++nodesGenerations_[index];
    }
}
```

Reusing a slot

```
struct MetaData
{
    int generation = 0;
    int skipfield = 0; // zero if active, non-zero to know how many field to skip
}

std::vector < Node > nodes_;
std::vector < MetaData > metadatas_;

void InsertNode(Index removedIndex)
{
    // Finding a non-zero skipfield

    // if no free slot we push back at the end
}
```

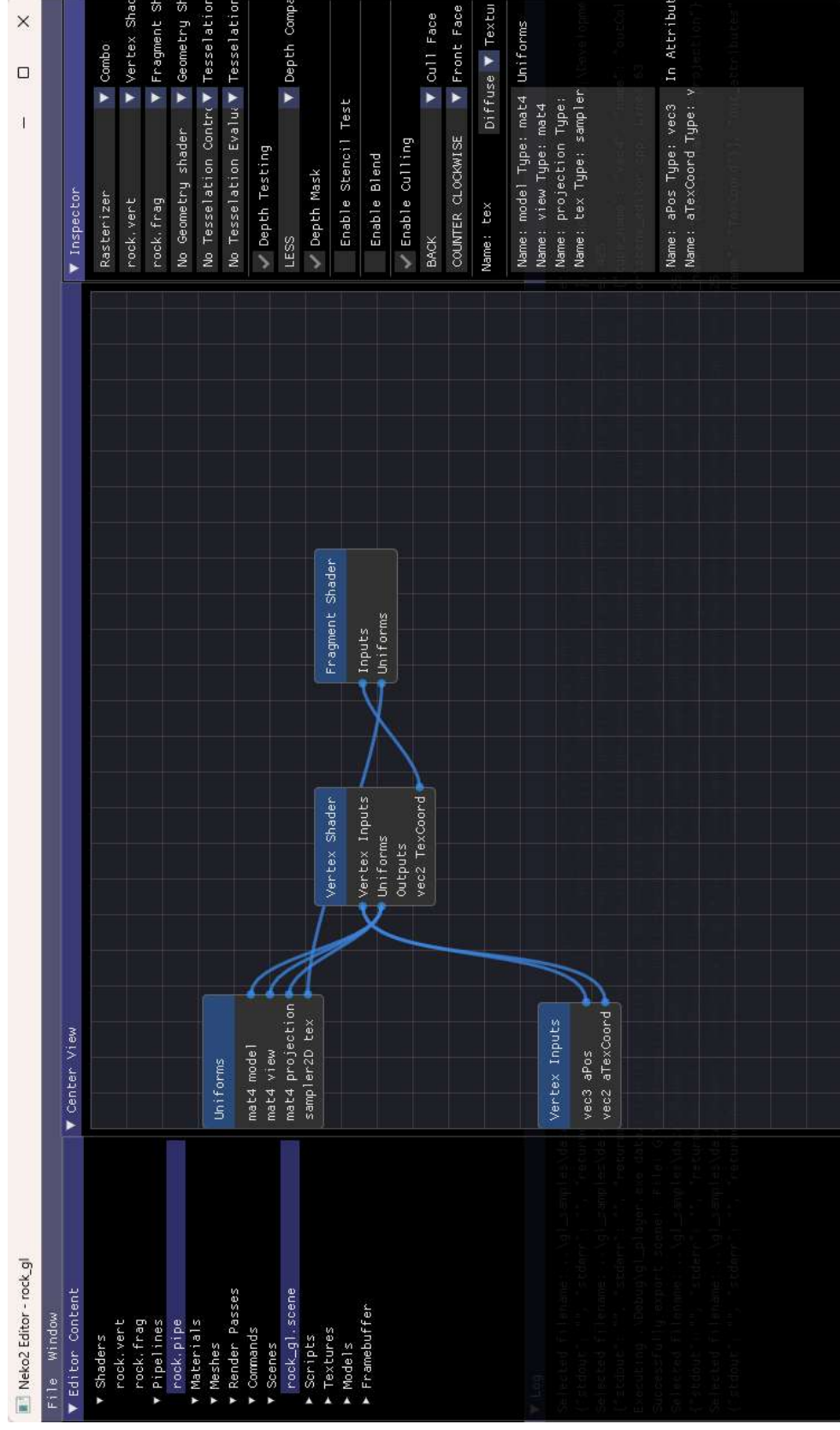


C++26 `std::hive` is a cousin of this pattern

- Different allocation strategy (no iterator invalidation for `std::hive`)
- `p1f::colony` has an `is_active(iterator)` method to check if the slot is used, not if the iterator is valid.
- Both have active/inactive objects

Graphics abstraction

Neko3d - My renderer



A brief history of Neko3d

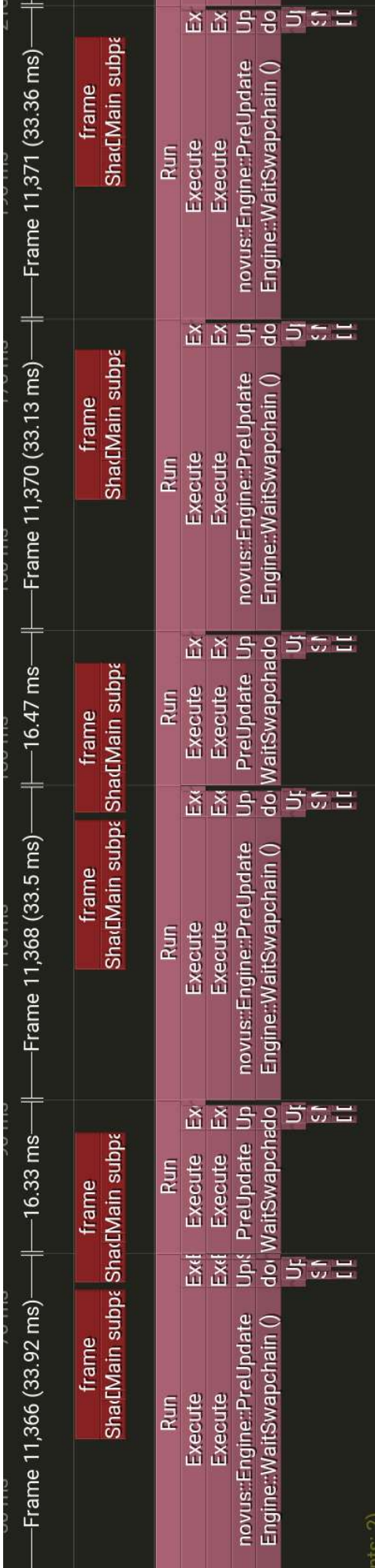
- Used as a “Computer Graphics Editor” for non-C++ tech art students in my courses
- Based on an OpenGL 3.0 ES backend with Python scripting
- Recently switched to SDL3 GPU and WASM scripting while sort of keeping the same API

Graphics abstractions

- Shaders: Small programs that runs on the GPU.
- Pipelines: Vertex + Fragment Shaders.
- Materials: Pipelines + Textures,
- Draw commands: Materials + Meshes
- Framebuffer: the render targets (aka where we are going to draw, usually a texture/the screen)
- Render passes: contain subpasses with a specific framebuffer and draw commands.

Rendering at 60 Hz on consoles

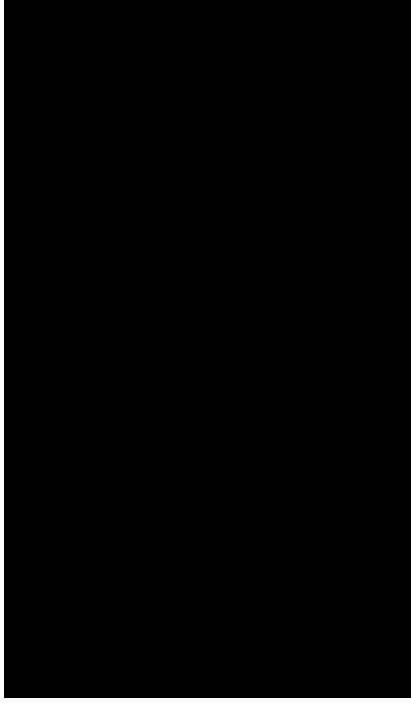
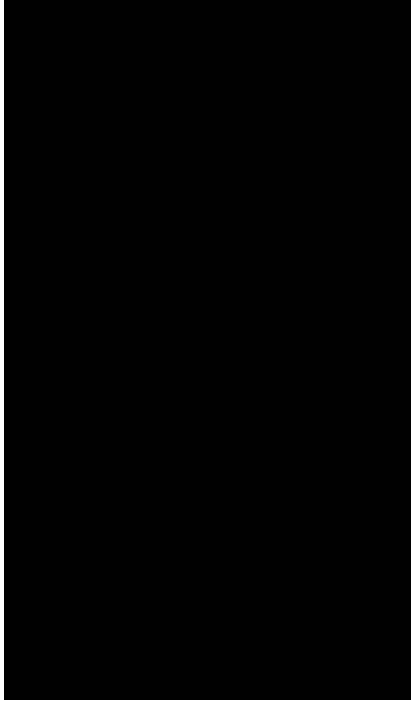
For my game to run at 60 Hz on the Switch, I need both the CPU frame and the GPU frame to be under 16.6 ms.



33.3ms when missing a vsync

Three render passes

Capture done with RenderDoc



1. Shadow depth

2. Depth prepass

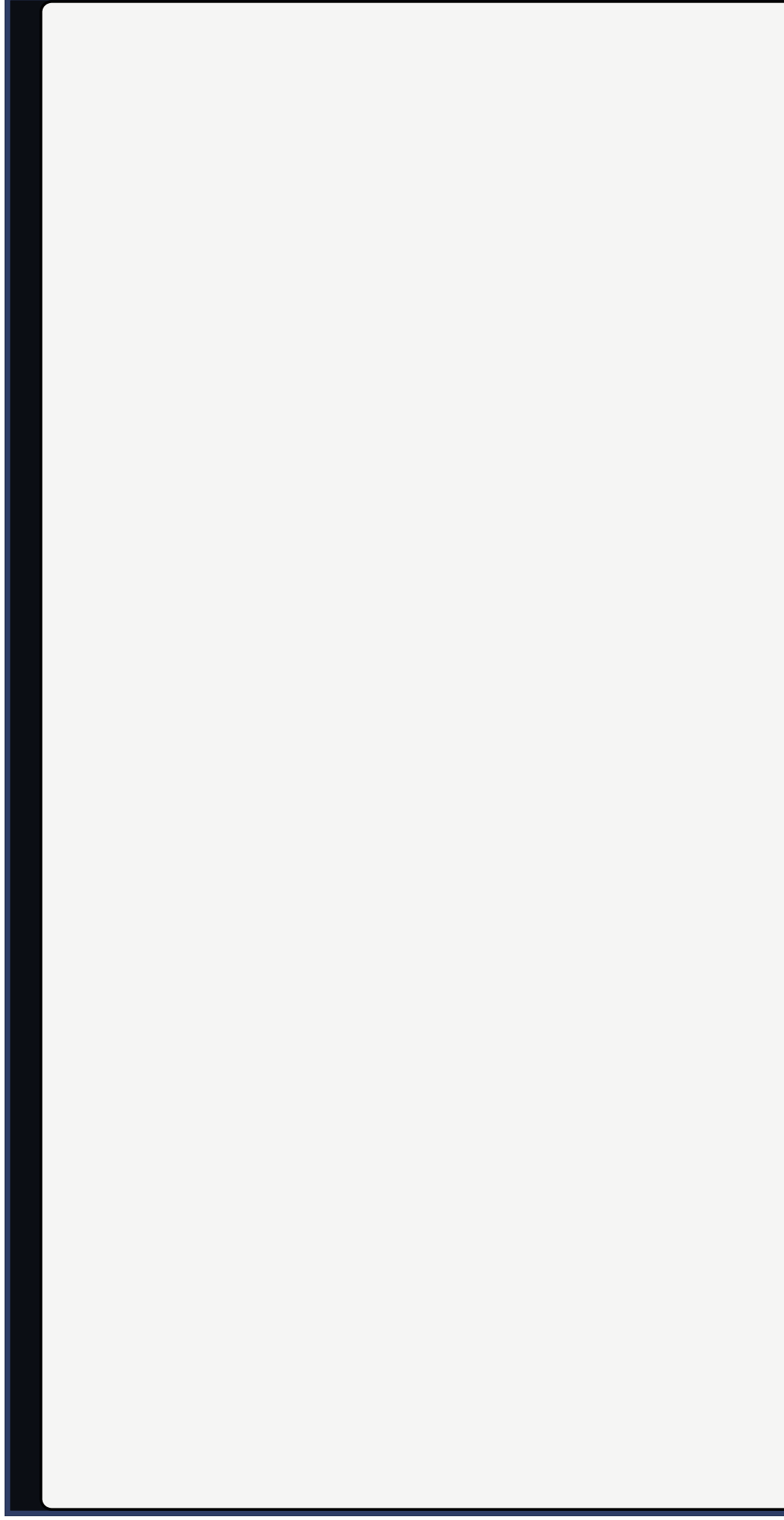
3. Color

Walkthrough demo - automated test



The snap rail system

Walkthrough demo in the browser?!?



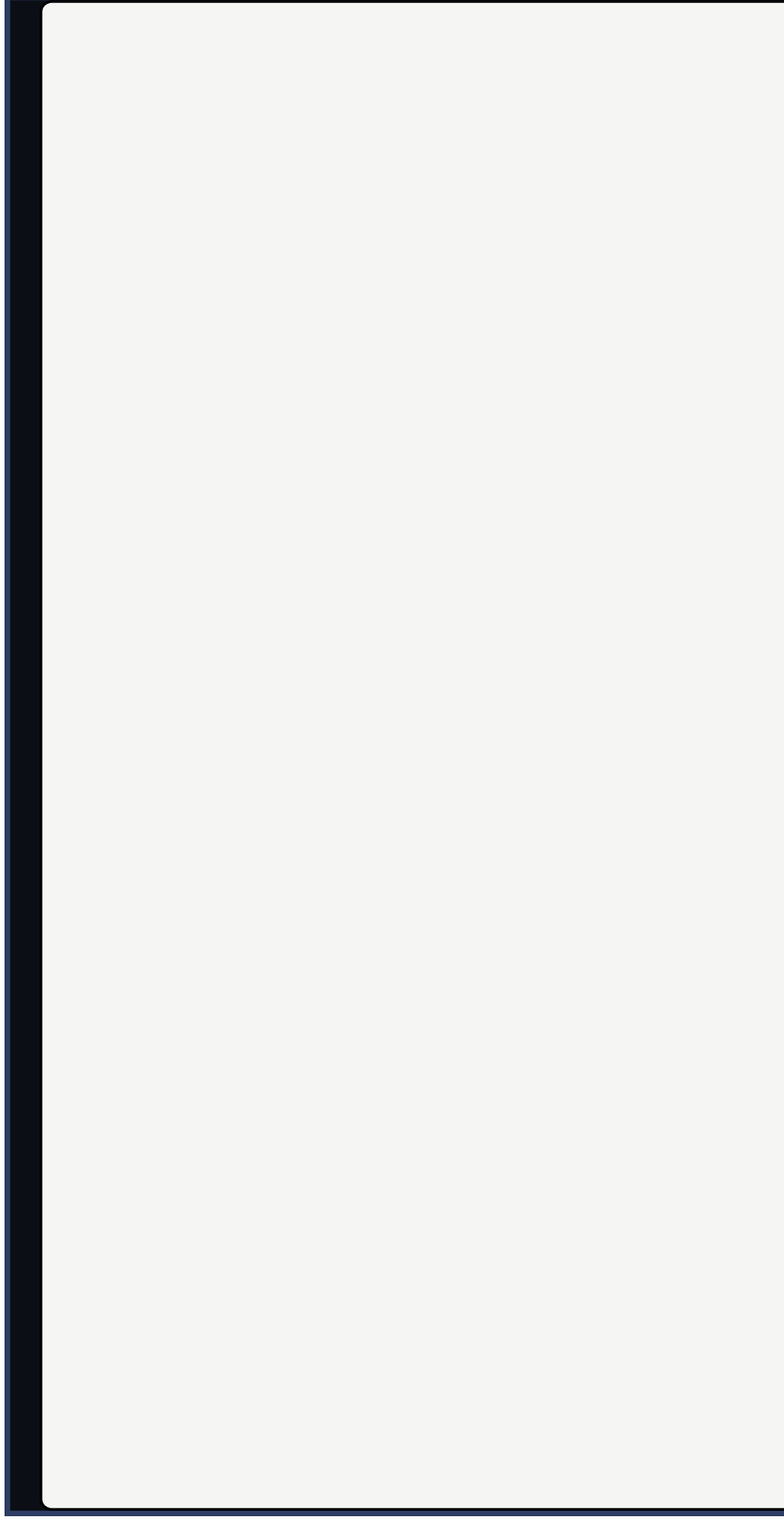
Tracy profiler

WHAT IS IT?

A real time, nanosecond resolution, remote telemetry, hybrid frame and sampling profiler for games and other applications.

A very nice feature of tracy is that I can mark CPU and GPU calls.

Not hitting 60Hz



Back-face culling

No need to draw the triangles at the back of an object from the point of view of the camera.

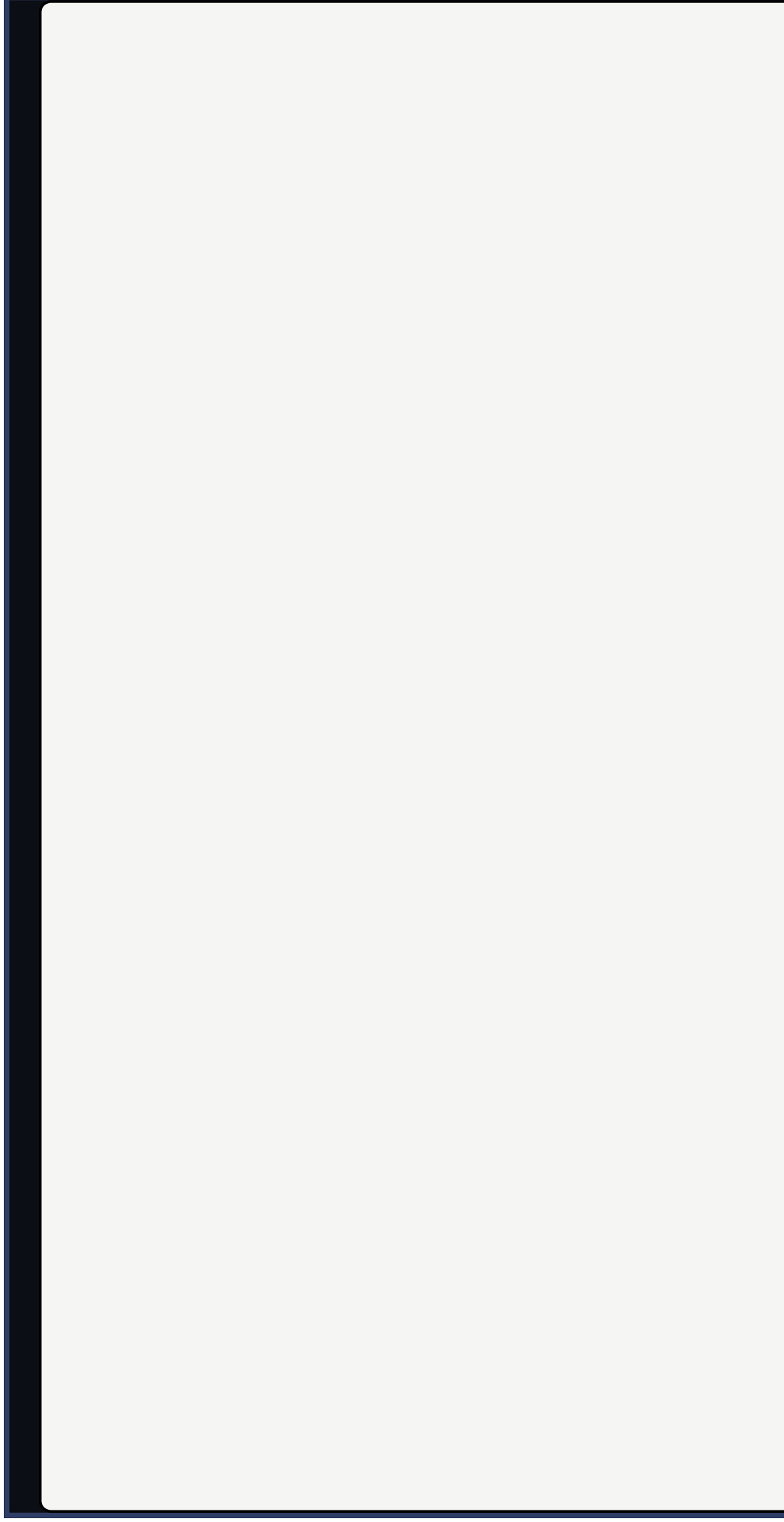
It's super easy to implement (a toggle in the graphics pipeline state). However, if your assets were not authored corrected, you might see some faces disappearing (I have a specific issue because of an Blender upgrade).

Frustum culling

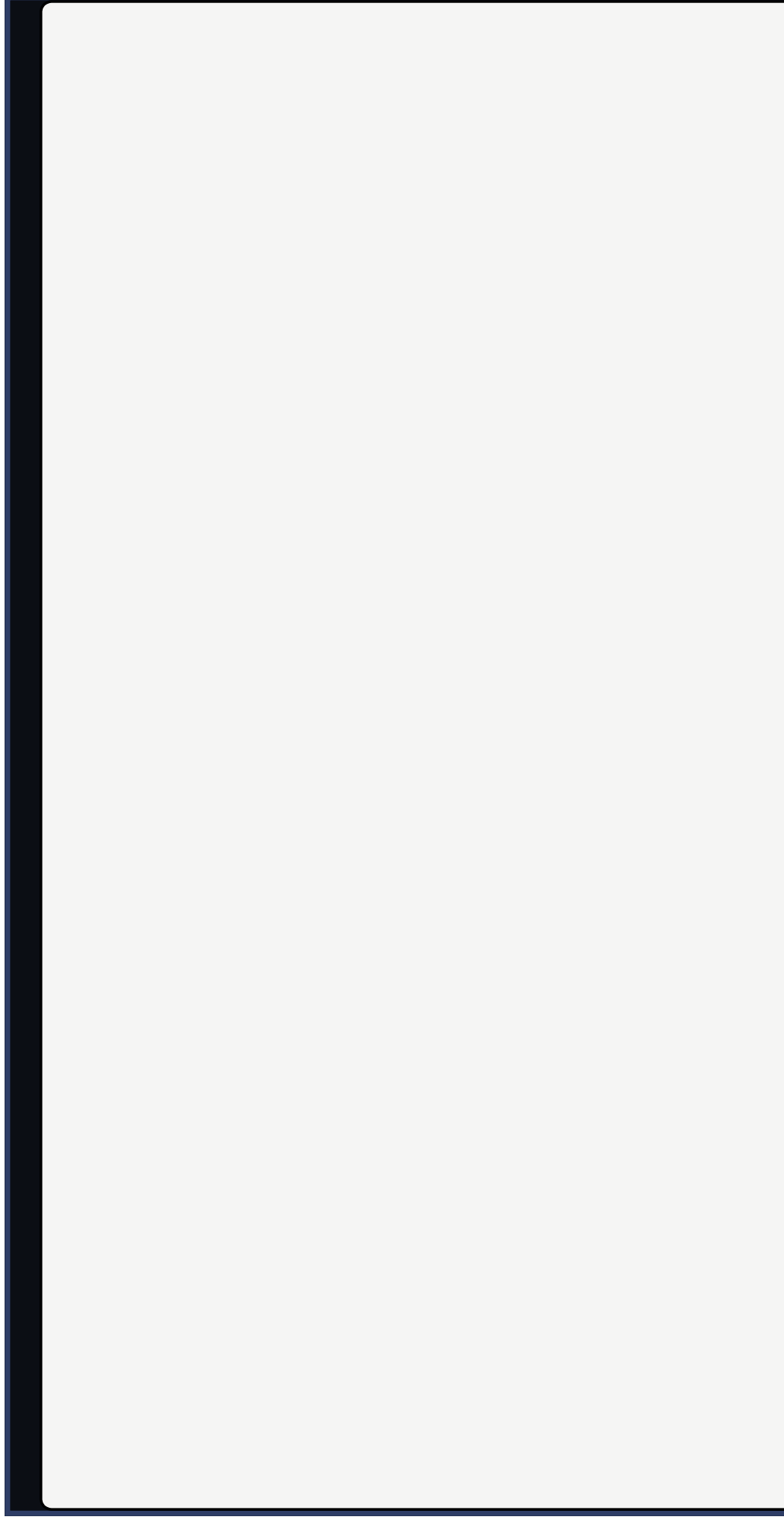
No need to draw objects that are outside of the view of the camera.



Culling in the browser



My profiler in the browser?!



Scene

Scene Tree

A scene tree is a hierarchical structure that a game engine uses to organize everything in a game scene.

One nice feature of a scene tree is that we can make an object the parent of other objects, so transforming the parent also transforms all its children.

```
struct SceneNode
{
    Index parent = -1;
    Index first_child = -1;
    Index next_sibling = -1;
```

Scene Node transform

```
// in scene_tree.h
struct SceneNodeInfo
{
    // [...]
    glm::vec3 position{0.0f};
    glm::vec3 scale{1.0f};
    glm::vec3 rotation{0.0f};
};
// ComputeLocalTransform () in scene_tree.cpp
auto transform = glm::mat4_cast(glm::quat(glm::radians(node.rotation)));
transform = glm::scale(glm::mat4(1.0f), node.scale) * transform;
transform = glm::translate(glm::mat4(1.0f), node.position) * transform;
```

ComputeLocalTransform() costs

Platform	Median	Observed range
Cortex-A57, Clang, -O3 -ffast-math	~470 ns	416–518 ns

One local matrix recompute, micro-benchmarked in isolation (median of 30 repetitions).

In game, recompute all on the docks (471 nodes): **160 μ s per frame**.

Dirty flags

Instead of recomputing every scene node's model transformation matrix, mark affected nodes as dirty:

```
transforms_[nodeIndex].dirty = true;
int32_t child = nodes_[nodeIndex].first_child;
while (child >= 0)
{
    MarkDirty(child);
    child = nodes_[child].next_sibling;
}
// in UpdateTransforms()
for (int32_t idx : depthFirstOrder_)
{
    auto& transform = transforms_[idx];
    if (!transform.dirty)
        continue;
}
```


The difference in numbers

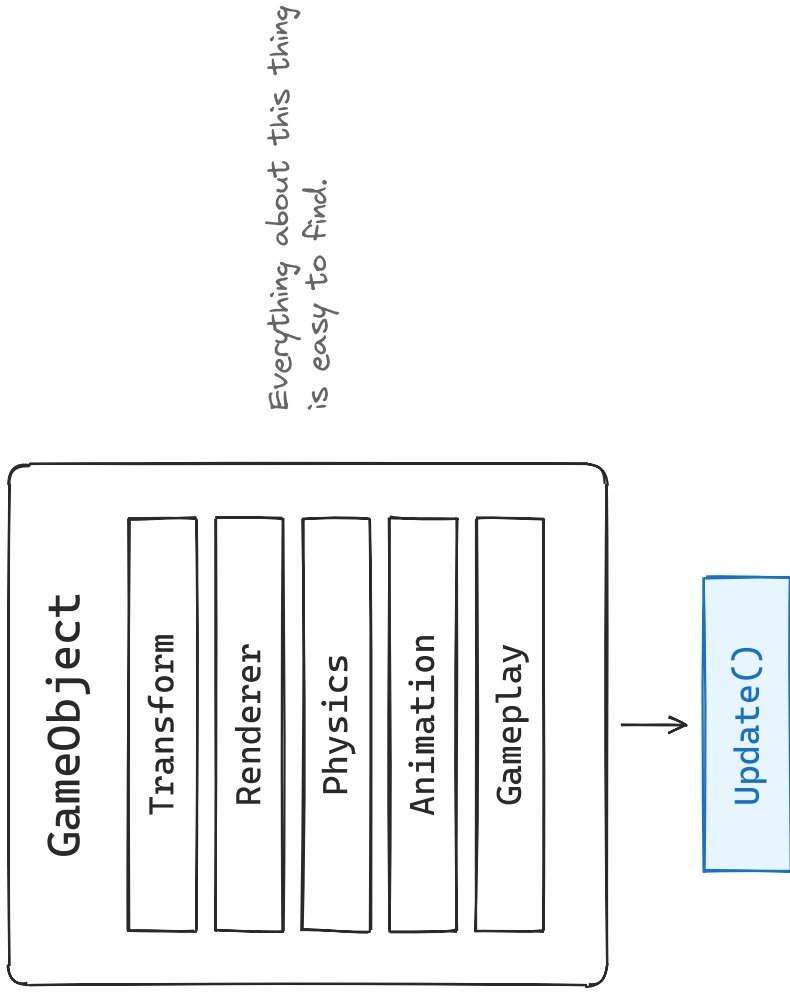
Level	Recompute all	Dirty flag	Set*
docks/rail00 (explore)	160.2 μ s	17.0 μ s	37 calls
palace/rail00 (explore)	119.9 μ s	11.6 μ s	45 calls
docks/seagulls_win_fight	162.4 μ s	19.2 μ s	221 calls

Includes the Set* [Position|Scale|Rotation] + UpdateTransforms times.

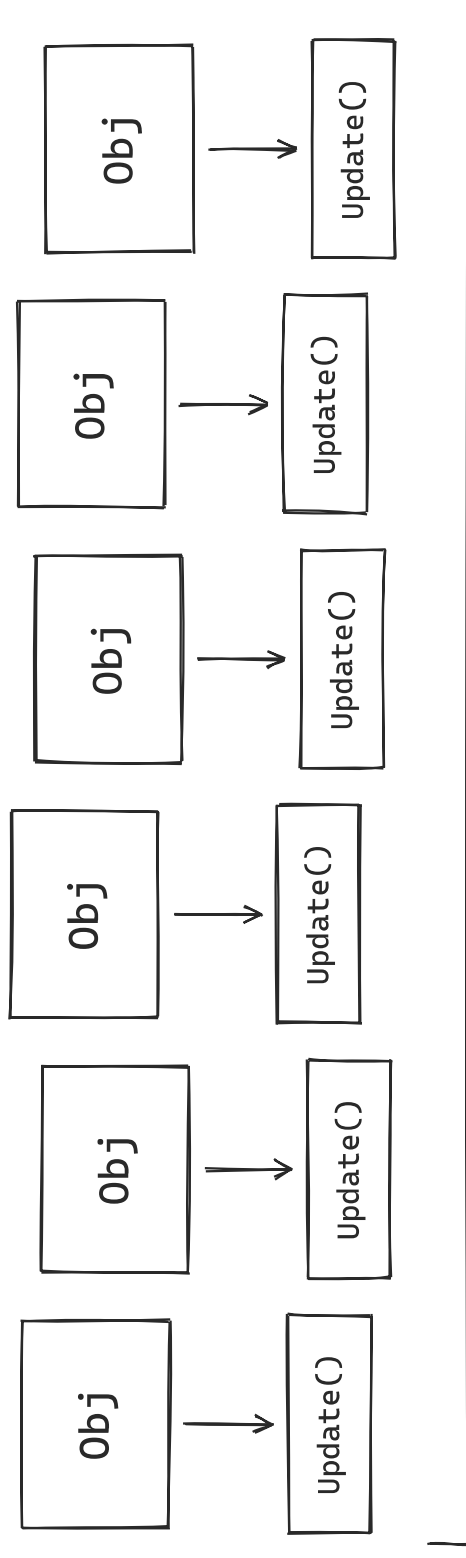
Game Engine Object Models

While a scene tree lets us organize relationships between entities, we still need a way to compose those entities from components.

Game Objects/Components Model



Lots of Game Objects



scattered updates
scattered data
harder to batch

Entity-Component-System Model

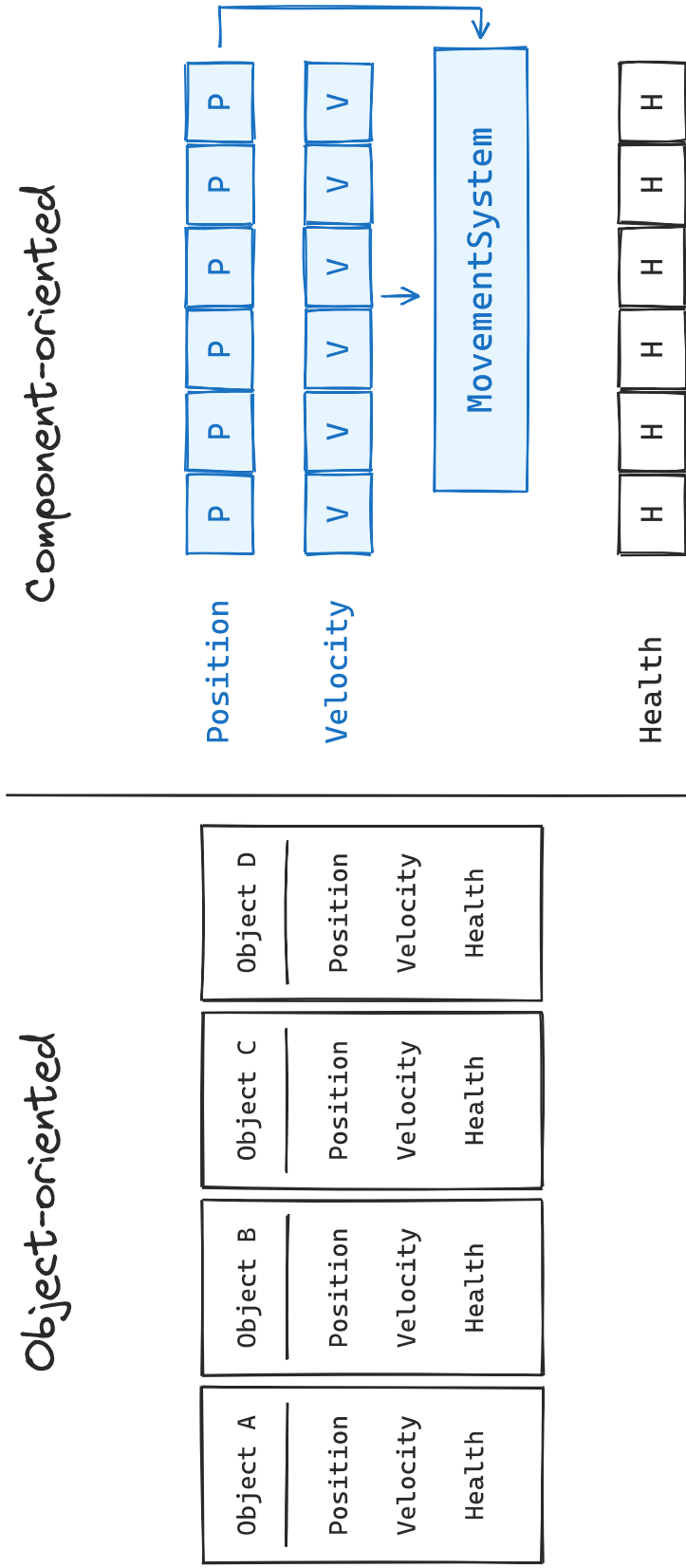
From Flems doc:

- Has *entities* that uniquely identify objects in a game
- Has *components* which are datatypes that can be added to entities
- Has *systems* which are functions that run for all entities matching a component query

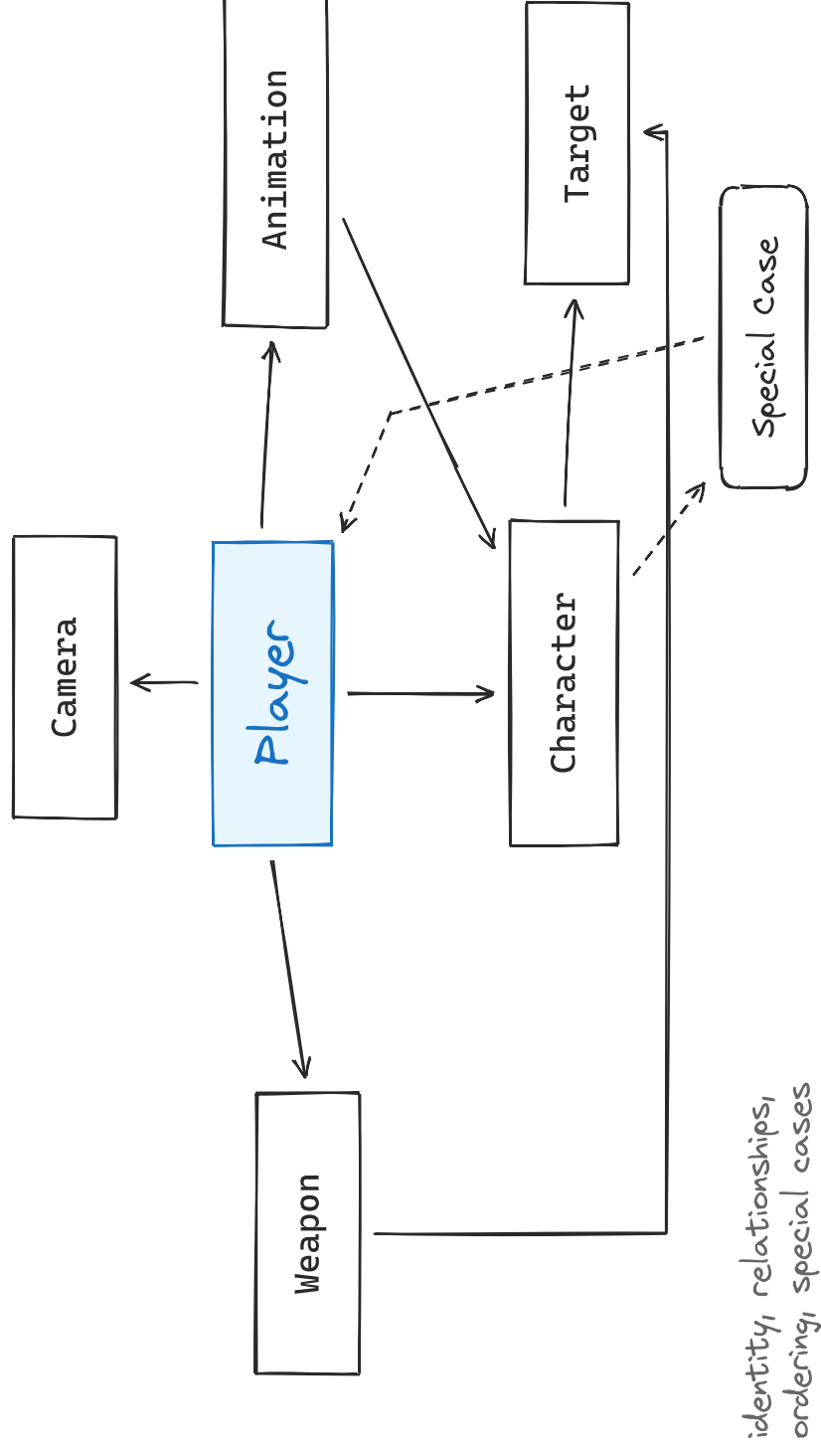
ECS in practice

```
struct Position { float x, y; };  
  
struct Velocity { float x, y; };  
  
MovementSystem(World& world)  
{  
    for (auto [position, velocity] :  
          world.query<position, velocity>())  
    {  
        position += velocity;  
    }  
}
```

Flipping the Game Object Model



But my game looks like this



Hybrid approach

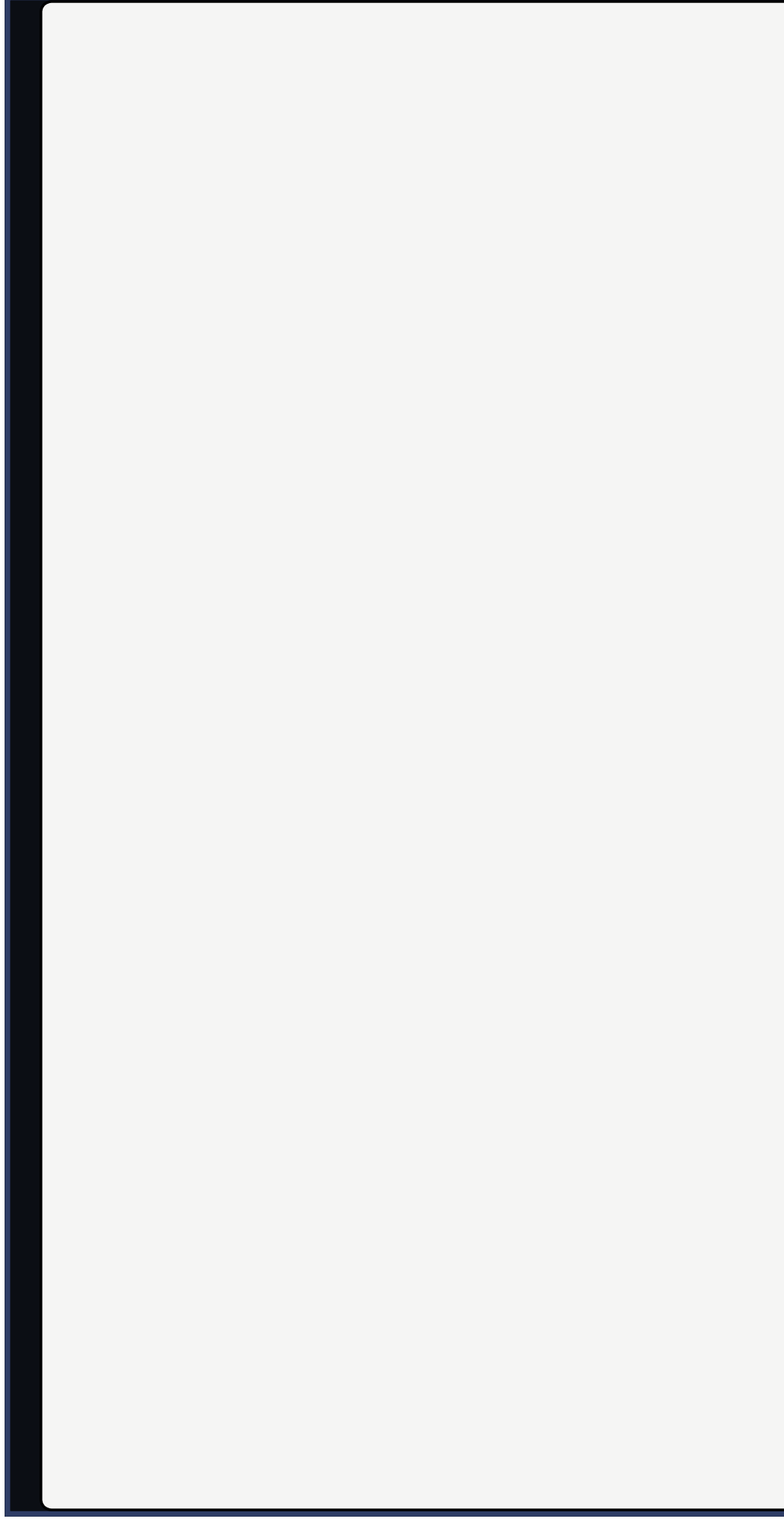
For Soup Raiders, I have systems that manages a subset of game objects at the same time, taking the best of both worlds.

Assets

Assets - What I got from my Unity project

- PNG/JPG for sprites/textures
- FBX exported from Blender
- JSON for gameplay data
- FMOD banks for sounds and music

Assets - what they look like



First load

To load from boot to the docks level (the biggest level in the demo), we are loading 74 images (including the 6 faces of the skybox) for a total of:

RESULT

11 692 ms (well above my sub-3s goal)

PNG is not a GPU format

Using `stb_image`, what we are doing is loading the PNG from disk to RAM, decompressing them and then uploading the decompressed RGBA8 version to the GPU.

KTX2/Basisu

Instead of decompressing a PNG, we are transcoding from the basisu universal to the specific compressed format supported on the platform (here ASTC 4x4) with those Basis Universal encoded data:

- ETC1S is used for opaque level art: 85 textures.
- UASTC (+ zstd 18) for cutout materials, atlases, skybox, water textures, etc...

Round two

Now loading KTX2 instead, 69 images (down from 74 PNG because the can actually put the whole skybox into one ktx2 texture) with 23.3 MB (down from 41.1 MB):

RESULT

7 404 ms (still above my sub-3s loading goal) ~1.5x faster than PNG.

Wait, aren't you loading from the main thread?

Yeah, it's bad!

- “Not Responding” window
- Hard requirements from the console to show something on the screen.
- Also the Switch has 4 cores, why not use them?

Job system

You might know it as a task scheduler, task graph, work scheduler, dispatcher, task pool, or an executor.

A system that schedules units of work on other threads while respecting (hopefully) each unit's requirements (dependencies, etc.).

Dividing the Load into Jobs

1. `PrepareDecode()` builds `decodeOrder_` (sorted most expensive first).
2. One `DecodeTextureWorkerJob` per worker (3 on the Switch) runs async on a pregenerated list of texture to import. It calls `DecodeOne()` in a loop which is: `core::LoadFile → ktxTexture2_TranscodeBasis`
3. `PumpTextureUpload` on the main thread, each frame it calls `CreateTexturesGPU` and `UploadTexturesGPU` on the textures that are already decoded, within a time budget.

Final Result (?)

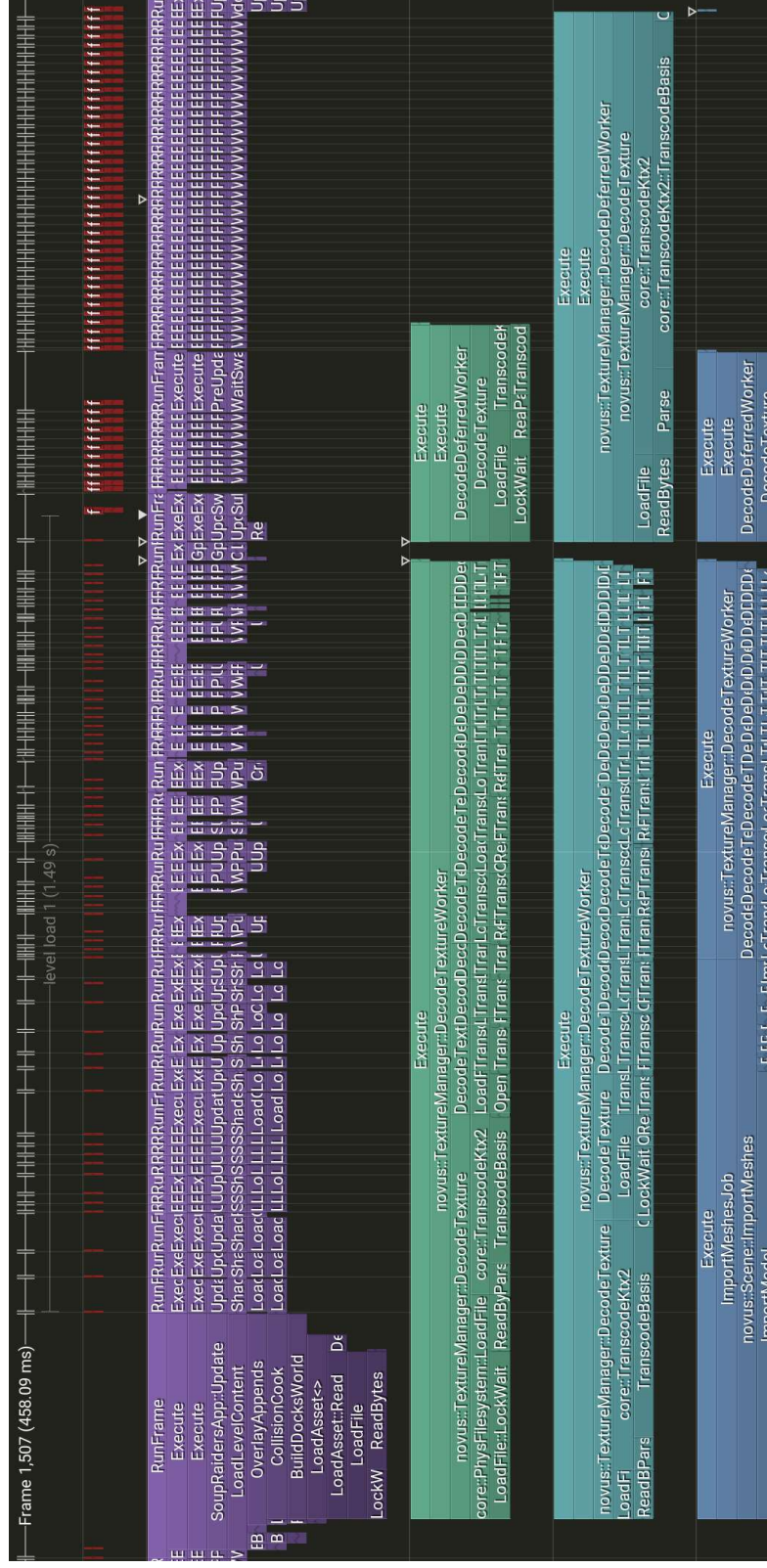
RESULT

3 113 ms

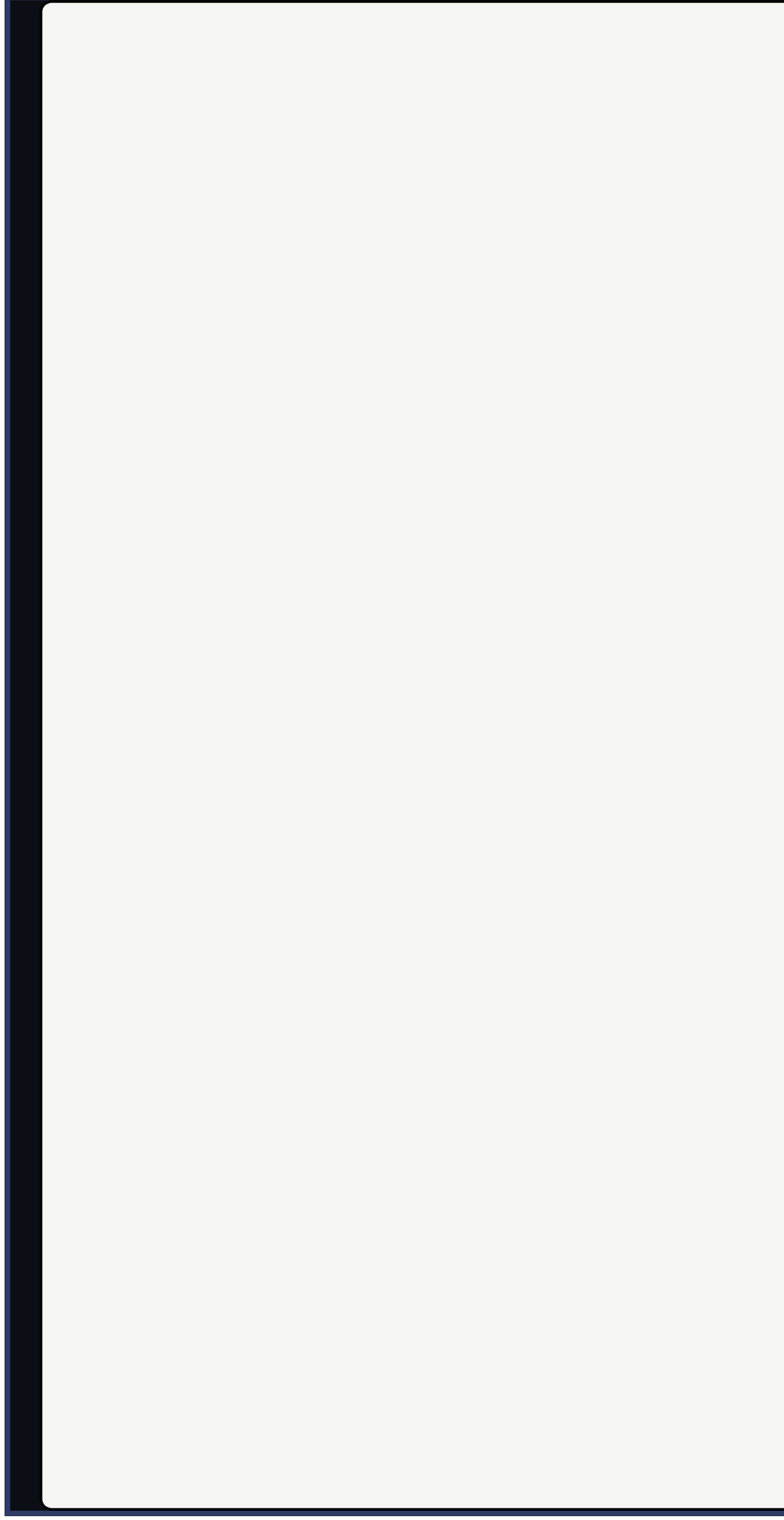
Did we fail?

Actually...

We are loading the battle scene ahead of time as well, but the game is already running, so technically the actual perceived loading time is ~2s.



My profiler in the browser, again



Serialization - A look at C++26 reflection

An example from Splash Online (CppCon 2024)

```
for(int playerNumber = 0; playerNumber < MaxPlayerNmb; playerNumber++)
{
    if(!IsValid(playerNumber))
    {
        continue;
    }
    const auto* player = reinterpret_cast< const std::uint32_t* >(
        &playerCharacters_[playerNumber]);

    for(std::size_t i = 0; i < sizeof(PlayerCharacter)/sizeof(std::uint32_t); i++)
    {
        result += player[i];
    }
}
```



The PlayerCharacter in question

```
//Respawn
Timer< neko::Scalar, 500> respawnPauseTimer{ neko::Scalar{ -1 }
Timer< neko::Scalar, 1000> respawnMoveTimer{ neko::Scalar{ -1 }
Timer< neko::Scalar, 2000> respawnStaticTime{ neko::Scalar{ 2 }
Timer< neko::Scalar, 2000> invincibleTimer{ neko::Scalar{ -1 }
int fallCount = 0;
int killCount = 0;

//Wata Hit
Timer< neko::Scalar, 1500> hitTimer{ neko::Scalar{ -1 }{};
neko::Vec2f hitDirection{};
int resistancePhase = 1; //Used for the resistance to wata bu
int hitPlayer = -1;
```

```
//Wata shoot
Timer< neko::Scalar, 4000> reserveWaterTimer{ neko::Scalar{ 4 }
int firstShots = FirstShotsCount;
Timer< neko::Scalar, 200> waterTimer{ neko::Scalar{ 0.3f }{};
Timer< neko::Scalar, 1033> reloadTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 100> jetBurstCoolDownTimer{ neko::Scalar{
Timer< neko::Scalar, 1000> jumpTimer{ neko::Scalar{ -1 }{};
Timer< neko::Scalar, 500> preJetBurstTimer{ neko::Scalar{ -1 }
//Dash
neko::SmallVector< neko::Vec2f, trailLength> dashPositions{};
Timer< neko::Scalar, 150> dashDownTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 150> stopDashTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 300> slowDashTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 150> bounceDashTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 500> dashPrepTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 500> dashedTimer{ neko::Scalar{ -1.0f }{}
Timer< neko::Scalar, 1000> wasDownRecoverTimer{ neko::Scalar{
Timer< neko::Scalar, 200> recoilTimer{ neko::Scalar{ -1.0f }{}
neko::Vec2f recoilDirection{};
Timer< neko::Scalar, 250> collidedTimer{ neko::Scalar{ -1.0f }{}
int collidedPlayer = -1;
int footCount = 0;
```

With C++26 reflection - a generic walker

```
//Expands the loop at compile time, producing code once for every
//reflected member rather than performing a normal runtime loop.
template for (constexpr auto member :
    // promotes that result to static storage
    // so it can be consumed by an expansion statement.
    std::define_static_array(
        // Returns reflections for T's non-static data members, in declaration
        std::meta::nonstatic_data_members_of(
            //Turns the type T into a compile-time std::meta::info describing
            ^T,
            std::meta::access_context::current()))
{
    mix_value(hash, object.[:member:]);
}
```

Reflection - what about Soup Raiders (or Neko3d)?

- Neko3d uses serialization/deserialization extensively because it used to be an editor, so you want to save/load.
- The first version used `ProtoBuf` because I have a friend who used to work at Google and told me it was great.
- Then I had allocation/performance concerns, so I switched to `FlatBuffers` (also Google).
- Eventually, I realized that I only needed to save my data to disk, rather than send it through gRPC, so I dropped the schema language and switched to `cereal`.

Reflection with cereal

```
struct RasterizerStateInfo
{
    float depth_bias_constant_factor = 0.0f;
    /* ... */
    CullMode cull_mode = CullMode::NONE;
    FrontFace front_face = FrontFace::COUNTER_CLOCKWISE;

    template < class Archive >
    void serialize(Archive& ar)
    {
        ar(CEREAL_NVP(depth_bias_constant_factor), /* ... */ ,
           CEREAL_NVP(cull_mode), CEREAL_NVP(front_face));
    }
};
```

The problem with `cereal` (for me)

- Exceptions: reports failure by throwing (not the best on consoles)
- Duplication of fields for both definition and reflection (can be a source of bugs).

So static reflection solves this, but...

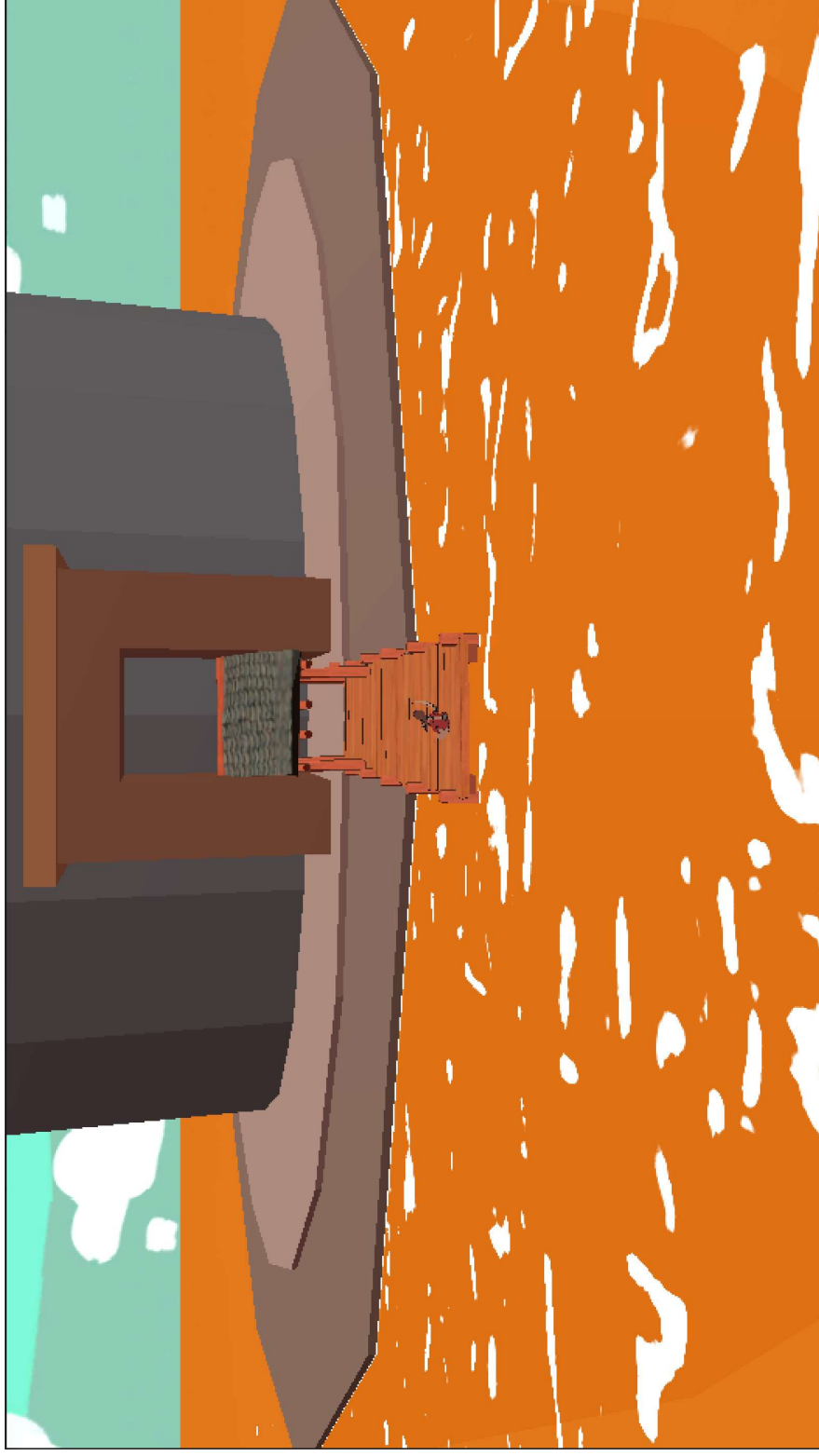
The problem with `glm::vec3`

`glm::vec3` is **three anonymous unions**: `union {T x, r, s;};`
`union {T y, g, t;}; union {T z, b, p;}`. A generic walker
recurses into each and emits **nine floats where cereal emits three**
(because the `serialize` function has to be written by hand anyway).

```
template < class Archive >
void serialize(Archive& archive, vec3& v)
{
    archive(cereal::make_nvp("x", v.x), cereal::make_nvp("y", v.y),
           cereal::make_nvp("z", v.z));
}
```

Conclusion

What's the future for Soup Raiders?



The Globe Island

Did I achieve my goal?

Yes! The full game is running at 60 Hz on the Switch, one comment that I got at the local gamedev meetup was: “The game looks smoother than the Unity version.”

Conclusion

- Containers: *Know your max sizes; most game data fits on the stack.*
- Handles: *Generational indices turn use-after-free into a check that fails.*
- Scene: *Dirty flags: measure, then skip the work.*
- Rendering: *Profile before optimizing; culling was one of the the big win.*
- Assets: *Convert at build time (KTX2), not at load time; then use jobs.*
- Reflection: *C++26 removes the duplicated field lists, but not third-party quirks like glm::vec3.*

Final quote by Tyler Glaiel

AN ENGINE IS NOTHING WITHOUT A GAME.

Make a game at the same time as you're making the engine. This is an unbreakable rule. The only unbreakable rule. Get the basics in as fast as you possibly can and then immediately start making a game on top of it.

References

- [Battlefield 6: Pushing visual fidelity while optimizing for all hardware - Graphics Programming Conference 2025](#)
- [Game Engine Entity/Object Models by Bobby Angelov](#)
- [Games Programming Pattern by Bob Nystrom](#)
- [Rollback System in C++ for Online Multiplayer Games - Elias Farhan - CppCon 2024](#)

Softwares/Libraries used

[RenderDoc](#)

[Tracy profiler](#)

[FMOD](#)

[SDL3 \(SDL GPU\)](#)

[GLM](#)

[stb_image](#)

[Basis Universal](#)

[KTX-Software \(libktx\)](#)

[Assimp](#)

[Protocol Buffers](#)

[FlatBuffers](#)

[cereal](#)

Complementary resources

- [C++ Data Structures That Make Video Games Go Round - Al-Afiq Yeong - CppCon 2024](#)
- [An Introduction to Tracy Profiler in C++ - Marcos Slomp - CppCon 2023](#)
- [Optimising Data Building In Game Development - Dominik Grabiec - ACCU 2025](#)
- [Why Every C++ Game Developer Should Learn SDL 3 Now - Mike Shah - CppCon 2025](#)

Thank you

QUESTIONS?

eliasfarhan.ch